NATIONAL TEACHERS COLLEGE
School of Teacher Education
629 J. Nepomuceno St., Quiapo, Manila

Reflection Question

19. Why does LIFO correctly match nested brackets?
20. Why must evaluation pop the right operand before the left operand?
21. What is the difference between balanced brackets and a valid arithmetic expression?
22. Compare your stack ADT with the function call stack. In a nested function call, which call returns first?
23. What memory is allocated and released by your implementation? Explain one resizing or node-deletion step.

Answer:

19. Nested brackets can be matched correctly using LIFO since the most recently opened bracket will always be placed on the top of the stack, and will be the first bracket retrieved and analysed when a closing bracket is encountered.

20. Listed in reverse order, a stack returns the values as 20th, 21st, 22nd, and so on, so the right-hand operand is on top and is the first to be popped; this is important to ensure operand order for non-commutative operations such as subtraction and division.

21. Balanced brackets only ensure that grouping symbols are properly paired and nested; a valid arithmetic expression is more complex; not only must the brackets be balanced, but the math rules also must be followed.

23. Both structures use the LIFO concept: Your Stack is a user-managed data structure stored on the heap, while the function call stack is managed by the CPU/OS - the most recently called function (the innermost call) completes and returns first.

23. The implementation dynamically allocates and deallocates heap memory using new T[] and delete[], respectively, and when resizing, allocates a new array with capacity twice that of the old one, copies the old one to the new one, and deletes the old one using delete[] data to avoid memory leaks.

NATIONAL TEACHERS COLLEGE
School of Teacher Education
629 J. Nepomuceno St., Quiapo, Manila